

# EtherCAT 主站开发完整技术规范

**适用环境:** Ubuntu 22.04 + Xenomai (Cobalt) + IgH EtherLab Master **目标硬件:** Hans Robot Elfin (基于 HCFAX3E.xml 配置) + INEXBOT IO + 末端 IO **核心特性:** XML 动态配置、Gap 内存对齐处理、双轴逻辑映射、硬实时控制

## 1. 总体架构设计 (System Architecture)

基于 Xenomai 实时框架，系统分为 **非实时域 (NRT - Linux Kernel)** 和 **实时域 (RT - Cobalt Kernel)**。

### 1.1 架构层级图

| 层级 | 名称          | 职责详解                                                                      | 运行域          | 关键技术                         |
|----|-------------|---------------------------------------------------------------------------|--------------|------------------------------|
| L4 | 应用与算法层      | 机器人学算法（正逆解）、轨迹规划、碰撞检测、业务状态机。                                              | RT (Primary) | C++ / Eigen                  |
| L3 | 设备抽象层 (API) | 提供面向对象的设备控制接口 ( ServoDrive , RobotArm , IoBoard )，屏蔽底层 PDO 映射细节。          | RT (Primary) | C++ Classes                  |
| L2 | 主站管理层       | 负责 Xenomai 实时线程管理、周期定时、域数据交换、XML 解析（仅初始化时）。                               | Mixed        | Xenomai POSIX Skin, TinyXML2 |
| L1 | 协议栈内核       | <b>IgH EtherLab Master</b> : 负责 EtherCAT 报文的组装、发送和接收，DC 时钟同步。             | Kernel       | ec_master 模块                 |
| L0 | OS/硬件层      | <b>Linux + Xenomai Cobalt</b> : 提供硬实时调度。网卡驱动使用 ec_generic 或 Xenomai 专用驱动。 | Hardware     | RTDM Driver / Generic        |

## 2. Xenomai 实时开发铁律 (Critical Rules)

在 Xenomai 环境下开发 L2 和 L3 层代码时，必须严格遵守以下规则，否则会导致严重的实时性问题（抖动、丢包）。

### 1. 零模式切换 (Zero Mode Switch):

- 定义:** 实时线程从 Cobalt 核（硬实时）切换回 Linux 核（软实时）的过程。
- 红线:** 在 start() 启动后的实时循环 ( rt\_thread\_func ) 内部，**严禁**调用任何 Linux 系统调用。
- 禁止列表:**
  - 标准 IO: printf , cout , fstream (替代方案: rt\_printf , 共享内存日志)。
  - 内存分配: malloc , free , new , delete (替代方案: 在 init 阶段预分配所有内存)。

- 时间函数: `gettimeofday` (替代方案: `clock_gettime(CLOCK_MONOTONIC)` )。
- 普通文件读写、Socket 通信。

## 2. 内存锁定 (Memory Locking):

- 程序启动时必须调用 `mlockall(MCL_CURRENT | MCL_FUTURE)` , 防止实时线程的内存页被 Linux 交换到磁盘 (Page Fault 是实时杀手)。

## 3. 栈预取 (Stack Prefaulting):

- 在创建线程后, 立即写入堆栈的每一页, 强制分配物理内存。

## 3. 硬件适配与 XML 解析 (Configuration & Parsing)

这是适配 Elfin 机器人及周边 IO 板卡的核心。Elfin 的 EtherCAT 从站 (HCFAX3E.xml) 包含大量内存对齐用的 Gap, 解析器必须精确处理。

### 3.1 真实 XML 结构参考

```
<EtherCATConfig>
  <Master cycle_time_us="1000" />

  <!-- Slave 0: INEXBOT IO Board (通用 IO) -->
  <Slave alias="0" position="0" name="Inexbot_IO" type="General_IO">
    <VendorID>0x00000025</VendorID> <!-- Decimal 37 -->
    <ProductID>0x00000530</ProductID>

    <!-- RxPDO (Output): 映射 DO -->
    <SyncManager index="2" dir="Output">
      <PDO index="0x1600" name="RxPDO-Map">
        <!-- 将 SubIndex 001 映射为友好的名字 -->
        <Entry index="0x7000" subindex="0x01" bitlen="32" name="DO_Port1"
        <Entry index="0x7000" subindex="0x02" bitlen="32" name="DO_Port2"
        <!-- ... -->
      </PDO>
    </SyncManager>
    <!-- TxPDO (Input): 映射 DI -->
    <SyncManager index="3" dir="Input">
      <PDO index="0x1A00" name="TxPDO-Map">
        <Entry index="0x6000" subindex="0x00" bitlen="32" name="DI_Port1"
        <Entry index="0x6001" subindex="0x00" bitlen="32" name="DI_Port2"
        <!-- ... -->
      </PDO>
    </SyncManager>
  </Slave>

  <!-- Slave 4: Elfin 关节模组 (J1 & J2) -->
  <Slave alias="0" position="4" name="Elfin_Joint_Mod_1" type="Elfin_Dual_Joint">
    <VendorID>0x0000001A</VendorID>
    <ProductID>0x50440200</ProductID>

    <!-- 逻辑轴定义: 用于 L3 层生成 API 前缀 -->
    <Axis id="1" name="Joint1" prefix="Axis1_" />
    <Axis id="2" name="Joint2" prefix="Axis2_" />

    <!-- RxPDO (主站 -> 从站) -->
    <SyncManager index="2" dir="Output">
      <PDO index="0x1600" name="RxPDO_Map_Axis1">
```

```

        <Entry index="0x6040" subindex="0x00" bitlen="16" name="Axis1_Con
        <Entry index="0x6060" subindex="0x00" bitlen="8" name="Axis1_Moc
        <!-- 关键：必须保留 Gap -->
        <Entry index="0x0000" subindex="0x00" bitlen="8" name="Gap_1"/>
        <Entry index="0x607a" subindex="0x00" bitlen="32" name="Axis1_Tar
        <Entry index="0x6071" subindex="0x00" bitlen="16" name="Axis1_Tar
        <Entry index="0x3097" subindex="0x00" bitlen="16" name="Axis1_Vel
        <!-- 大块 Padding -->
        <Entry index="0x0000" subindex="0x00" bitlen="160" name="Gap_Padc
    </PDO>
    <!-- 0x1610 Axis 2 配置同上，略 -->
</SyncManager>
<!-- TxPDO (从站 -> 主站) -->
<SyncManager index="3" dir="Input">
    <PDO index="0x1A00" name="TxPDO_Map_Axis1">
        <Entry index="0x6041" subindex="0x00" bitlen="16" name="Axis1_Stc
        <Entry index="0x6077" subindex="0x00" bitlen="16" name="Axis1_Act
        <Entry index="0x6064" subindex="0x00" bitlen="32" name="Axis1_Act
        <!-- ... 其他变量 ... -->
        <Entry index="0x0000" subindex="0x00" bitlen="8" name="Gap_2"/>
    </PDO>
    <!-- 0x1A10 Axis 2 配置同上，略 -->
</SyncManager>
</Slave>

<!-- Slave 7: Elfin End IO (末端 IO) -->
<Slave alias="0" position="7" name="Elfin_End_IO" type="Elfin_End_IO">
    <VendorID>0x0000001A</VendorID>
    <ProductID>0x10003201</ProductID>
    <!-- 映射末端特有的 IO 和 Modbus 寄存器 -->
    <SyncManager index="2" dir="Output">
        <PDO index="0x1600" name="End_RxPDO">
            <Entry index="0x7001" subindex="0x01" bitlen="16" name="End_Digit
            <Entry index="0x7002" subindex="0x01" bitlen="16" name="End_Modbu
            <!-- ... -->
        </PDO>
    </SyncManager>
    <SyncManager index="3" dir="Input">
        <PDO index="0x1A00" name="End_TxPDO">
            <Entry index="0x6001" subindex="0x01" bitlen="16" name="End_Digit
            <Entry index="0x6002" subindex="0x01" bitlen="16" name="End_Analc
            <!-- ... -->
        </PDO>
    </SyncManager>
</Slave>
</EtherCATConfig>

```

### 3.2 解析与配置逻辑

在 EtherCatMaster::init() 中执行 (非实时阶段):

1. **解析 XML:** 读取所有 <Entry> 。
2. **构建** ec\_pdo\_entry\_info\_t :
  - 对于普通 Entry: 填入 XML 中的 Index, SubIndex, BitLen。
  - 对于 Gap ( #x0000 ): 必须填入数组, IgH 需要它来占位。 {0x0000, 0x00, bitlen} 。

3. 调用 `ecrt_slave_config_pdos()` : 传入包含 Gap 的完整数组。

4. 注册 **Offset** ( `ecrt_slave_config_reg_pdo_entry` ):

- **跳过 Gap**: 循环遍历 Entry 列表, 如果 `Index == 0x0000` , 则跳过注册步骤。
- **记录有效 Offset**: 将非 Gap 变量的 Offset 存入 `m_offset_map` 。

## 4. 详细 API 接口设计 (API Reference)

### 4.1 主站管理器 (EtherCatMaster)

负责 Xenomai 线程管理和全局资源。

```
#include <pthread.h>
#include <sys/mman.h>
#include <rtdk.h> // Xenomai rt_printf

class EtherCatMaster {
private:
    pthread_t m_thread;
    bool m_running = false;
    ec_master_t* m_master = nullptr;
    ec_domain_t* m_domain = nullptr;
    uint8_t* m_domain_data = nullptr;

    // 实时线程函数
    static void* rt_thread_func(void* arg) {
        EtherCatMaster* ctx = (EtherCatMaster*)arg;

        // 标记为实时任务
        pthread_setname_np(pthread_self(), "EtherCAT_Loop");

        struct timespec wakeup_time;
        clock_gettime(CLOCK_MONOTONIC, &wakeup_time);

        // 激活主站
        ecrt_master_activate(ctx->m_master);

        while (ctx->m_running) {
            // 1. 周期等待 (Xenomai Hard Real-time Sleep)
            wakeup_time.tv_nsec += 1000000; // 1ms
            if (wakeup_time.tv_nsec >= 1000000000) {
                wakeup_time.tv_nsec -= 1000000000;
                wakeup_time.tv_sec++;
            }
            clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &wakeup_time, NULL);

            // 2. 数据接收
            ecrt_master_receive(ctx->m_master);
            ecrt_domain_process(ctx->m_domain);

            // 3. 用户回调 (L3/L4 逻辑)
            if (ctx->m_control_cb) ctx->m_control_cb();

            // 4. 数据发送
            ecrt_domain_queue(ctx->m_domain);
            ecrt_master_send(ctx->m_master);
        }
    }
};
```

```

        return NULL;
    }

public:
    // 初始化 (NRT)
    bool init(const std::string& xml_file) {
        mlockall(MCL_CURRENT | MCL_FUTURE); // 必须!
        rt_print_auto_init(1); // 初始化 Xenomai 打印

        m_master = ecrt_request_master(0);
        m_domain = ecrt_master_create_domain(m_master);

        // 解析 XML 并配置 PDO (此处包含上述 Gap 处理逻辑)
        // ...
        return true;
    }

    // 启动实时线程 (NRT -> RT)
    bool start() {
        m_running = true;
        pthread_attr_t attr;
        pthread_attr_init(&attr);
        pthread_attr_setschedpolicy(&attr, SCHED_FIFO); // 关键策略
        struct sched_param param = { .sched_priority = 90 }; // 高优先级
        pthread_attr_setschedparam(&attr, &param);

        pthread_create(&m_thread, &attr, rt_thread_func, this);
        return true;
    }
};

```

## 4.2 伺服驱动器 (ServoDrive) - L3

封装单轴逻辑。针对 Elfin 的双轴模块，我们通过构造函数传入前缀（如 "Axis1\_"）来区分。

```

class ServoDrive {
private:
    std::string m_prefix; // "Axis1_" 或 "Axis2_"
    EcSlave* m_slave;      // 指向物理从站对象的指针

public:
    ServoDrive(EcSlave* slave, std::string prefix) : m_slave(slave), m_prefix(pre

    // --- 实时控制接口 (RT Safe) ---

    // 设置目标位置 (CSP)
    void set_target_position(double pos_user_unit) {
        // 自动拼接: "Axis1_TargetPosition"
        // 内部查找 Offset 并写入 m_domain_data
        m_slave->write_pdo(m_prefix + "TargetPosition", convert(pos_user_unit));
    }

    // 设置速度前馈 (Elfin 专用)
    void set_velocity_feedforward(double vel_user_unit) {
        // Elfin 协议: 写入 VelFF (0x3097 或 0x3897)
        // 计算公式参考 PDF: VelFF = CmdVel * EncRes * Gear / 360 * 0.00008 * 1.025
        int16_t ff_val = calculate_elfin_ff(vel_user_unit);
    }
};

```

```

        m_slave->write_pdo(m_prefix + "VelFF", ff_val);
    }

    // 读取实际位置
    double get_actual_position() {
        return convert_back(m_slave->read_pdo<int32_t>(m_prefix + "ActualPosition"));
    }

    // CiA402 状态机切换 helper
    void set_control_word(uint16_t cw) {
        m_slave->write_pdo(m_prefix + "Controlword", cw);
    }

    uint16_t get_status_word() {
        return m_slave->read_pdo<uint16_t>(m_prefix + "Statusword");
    }
};

```

### 4.3 机械臂组 (RobotArmGroup) - L3

负责多轴协调和 Elfin 特有的复杂逻辑（如抱闸）。

```

class RobotArmGroup {
private:
    std::vector<ServoDrive*> m_joints; // 6个逻辑轴

public:
    // --- 抱闸控制逻辑（根据 PDF 文档 V2.0.4） ---
    // 注意：抱闸控制涉及 SDO 操作，建议在非运动状态下调用。
    // Xenomai 环境下调用 SDO 可能会导致模式切换，需谨慎。
    void release_brake(int joint_index) {
        // 1. 切换到手动开抱闸模式 (Mode = 11)
        // m_joints[joint_index]->set_mode(11);

        // 2. 发送开抱闸指令 (SDO 0x3023 或 0x3033)
        // 下发 0x11000000 -> 0x33000000 -> 0x22000000 序列
        // 参考 PDF 第 11 页
        // 注意：这里需要调用 sdo_write，这是非实时操作，只能在初始化或静止时做。
    }

    // --- 同步运动控制 ---
    void set_joint_positions(const std::vector<double>& positions) {
        if (positions.size() != 6) return;
        for(int i=0; i<6; ++i) {
            m_joints[i]->set_target_position(positions[i]);
            // 可选：计算速度并设置前馈
        }
    }
};

```

### 4.4 通用 IO 控制板 (GeneralIoBoard) - L3

针对 INEXBOT (Slave 0) 这种包含多个 32 位/16 位端口的 IO 板卡。

```

class GeneralIoBoard {
private:
    EcSlave* m_slave;

public:
    GeneralIoBoard(EcSlave* slave) : m_slave(slave) {}

    // --- 输出控制 (RT Safe) ---

    // 直接设置整个端口的值 (32位)
    // port_name: 对应 XML 中的 "DO_Port1"
    void set_output_port(const std::string& port_name, uint32_t value) {
        m_slave->write_pdo(port_name, value);
    }

    // 设置某一位 (读取-修改-写入)
    // 注意: 在多线程环境下可能需要锁, 但在单实时线程中是安全的
    void set_output_bit(const std::string& port_name, int bit_index, bool state)
        // 1. 读取当前缓存值 (这里需要 EcSlave 支持缓存或通过 read_pdo 读取)
        // 2. 修改对应 Bit
        // 3. 写入 PDO
        // 注意: 如果 DO 是只写 (RxPDO), 则需要 L3 层自己维护一个 shadow buffer
    {

    // --- 输入读取 (RT Safe) ---

    // 读取整个端口
    uint32_t get_input_port(const std::string& port_name) {
        return m_slave->read_pdo<uint32_t>(port_name);
    }

    // 读取某一位
    bool get_input_bit(const std::string& port_name, int bit_index) {
        uint32_t val = get_input_port(port_name);
        return (val >> bit_index) & 1;
    }
};

```

#### 4.5 末端 IO 模块 (ElfinEndIO) - L3

针对 Elfin 机械臂末端特有的 IO 模块 (Slave 7), 封装 PDF 第 2 章的功能。

```

class ElfinEndIO {
private:
    EcSlave* m_slave;

public:
    ElfinEndIO(EcSlave* slave) : m_slave(slave) {}

    // --- 数字 IO 控制 ---
    // bit 0-2: 数字输出 (24V)
    void set_digital_outputs(uint8_t mask) {
        // 读取当前值以保留 LED 状态 (bit 12-14), 或维护 shadow buffer
        // 写入 "End_DigitalOutputs"
        m_slave->write_pdo("End_DigitalOutputs", (uint16_t)mask);
    }
}

```

```

// --- LED 灯带控制 (Bit 12-14) ---
// 颜色代码: 0x1=绿, 0x2=红, 0x3=黄, 0x4=蓝 ... (参考 PDF 第 18 页)
void set_led_color(int color_code) {
    uint16_t val = (color_code & 0x7) << 12;
    // 写入时需要与 DO 状态进行位或运算
    // m_slave->write_pdo("End_DigitalOutputs", val | current_do_state);
}

// --- 模拟量读取 ---
float get_analog_input_ch1() {
    uint16_t raw = m_slave->read_pdo<uint16_t>("End_AnalogInput1");
    // 根据传感器转换电压值
    return raw * (10.0f / 4096.0f); // 假设 0-10V 12bit
}
};

```

## 5. Elfin 机器人专用控制逻辑详解

根据你提供的 PDF 文档，以下是 Elfin 机器人的核心控制参数：

### 5.1 抱闸控制 (Brake Control)

- **模式:** 必须将 Modes of operation (0x6060/0x6860) 设置为 **11** (自定义模式)。
- **SDO 地址:**
  - 通道1 (Axis1): **0x3023**
  - 通道2 (Axis2): **0x3033**
- **时序:**
  1. 写入 0x11000000 -> 打开抱闸。
  2. 等待反馈 (读取 0x2023/0x2033 等于 0x300000)。
  3. 写入 0x33000000 -> 清除状态。
  4. 写入 0x22000000 -> 关闭抱闸。
  5. 写入 0 -> 结束。
- **注意:** 抱闸操作完成后，记得把 Mode 切回 **8 (CSP)**。

### 5.2 速度前馈 (Velocity Feedforward)

- **PDO 对象:** VelFF (0x3097 / 0x3897)。
- **必要性:** 文档指出“跑高速时需要加上速度前馈，否则会报位置跟随错误”。
- **计算公式:**  $\text{velocityFeedforward} = \text{CmdVelAcs} * \text{EncResolution} * \text{GearRatio} / 360 * 0.00008 * 1.025$ 
  - CmdVelAcs : 指令角速度 (度/秒)。
  - EncResolution : 17位编码器 (131072)。
  - GearRatio : 减速比 (需查表, J1/J2=121.02 等)。



## 5.3 力矩模式 (Torque Mode)

- **限制:** PDF 第 13 页指出, 进入力矩环需要同一个从站的 `0x6060` (Axis1) 和 `0x6860` (Axis2) 同时下发 `0x0A` (10)。
- **实现:** 在 `RobotArmGroup` 中, 在同一帧 EtherCAT 报文中同时写入这两个变量。

## 6. 开发与调试路线图 (Roadmap)

### 第一阶段: Xenomai 环境验证

1. 安装打好补丁的 Linux 内核。
2. 运行 `/usr/xenomai/bin/latency`, 确保 **Worst case latency < 20us**。
3. 编译 IgH Master, 确保 `dmesg` 中显示 EtherCAT 主站已加载。

### 第二阶段: XML 解析与 PDO 映射

1. 编写 XML 解析器, 加载 `HCFA3E.xml`。
2. **Debug:** 打印解析出的 `ec_pdo_entry_info_t` 列表, 与 XML 文件逐行比对, 确保 Gap 一个不少。
3. 运行主站, 执行 `ethercat pdos`, 检查输出是否与预期一致。

### 第三阶段: 单轴开环测试

1. 不接机械臂, 仅接控制器。
2. 启动实时线程, 监控 `/proc/xenomai/sched/stat`。确保你的线程 `MSW` (Mode Switches) 计数器 **保持为 0** (除了启动瞬间)。这是 Xenomai 成功的标志。
3. 如果 `MSW` 增加, 检查是否在循环里用了 `printf` 或 `new`。

### 第四阶段: Elfin 联调

1. 连接 Elfin 机械臂。
2. 实现 `CiA402` 状态机, 使能 `Joint1`。
3. **抱闸测试:** 在 `PreOp` 状态下尝试通过 `SDO` 打开抱闸, 听到“咔哒”声。
4. **CSP 运动:** 发送正弦波位置指令。
5. **前馈调优:** 如果高速运动报错 `0x0080` (位置跟随误差), 检查 `VelFF` 计算公式。

## 7. 故障排查指南 (Troubleshooting)

### 1. Sync Error (同步丢失):

- 现象: 驱动器报警, IgH 日志显示 "Domain 0: Working counter changed to 0/1"。
- 原因: Xenomai 实时线程抖动过大, 或 DC 时钟未配置。
- 解决: 检查 `latency` 测试结果; 代码中确保调用了 `ecrt_master_application_time` 更新主站时间。

### 2. 数据错乱 (Data Corruption):

- 现象: 读到的位置值是天文数字, 或者 StatusWord 不对。
- 原因: XML 解析忽略了 #x0000 Gap, 导致 Offset 偏移。
- 解决: 重新核对 config.xml 和代码中的 PDO 结构体定义。

### 3. 模式切换 (Mode Switches):

- 现象: 实时性偶尔变差, MSW 计数增加。